

CORRIGÉ – Lab S03 – Tables de hachage

Structures de Données et Algorithmes Avancés – Master 1 GLSI

Dr. El Hadji Bassirou TOURÉ – ESP/UCAD – 2025–2026 • **Document prof – Réponses en bleu**

Ce document ne comporte pas de barème : le lab n'est pas noté. Il contient les réponses attendues et des notes pédagogiques à évoquer lors de la restitution en plénière.

Partie I – Pratique : observations attendues

Vérification 1 (tests JUnit, étape 4) – Les 5 tests doivent passer. Le test `testForcerUneCollision` est conceptuellement central : il prouve que le chaînage fonctionne grâce à `equals`, même quand le `hashCode` est identique.

À souligner en plénière : c'est la première fois que les étudiants voient que `hashCode` seul ne suffit pas — c'est le couple (`hashCode`, `equals`) qui distingue les clés. Cela prépare l'exercice théorique 5.

Vérification 2 (test du rehash, étape 5) – La capacité doit passer de 8 à 16 au moment de la 7^e insertion. Toutes les clés doivent rester accessibles.

Piège fréquent : un étudiant qui a oublié le cast (`double`) verra que la capacité ne change jamais. C'est un bug classique à provoquer et discuter — la division entière Java est un piège récurrent.

Vérification 3 (programme d'observation, étape 6) – réponses attendues :

1. **3 rehashs pendant 30 insertions**, aux moments $n = 7$, $n = 13$, $n = 25$.
2. Juste après chaque rehash, $\alpha \approx 0,4$ (entre 0,39 et 0,44).
3. **Non, α ne dépasse jamais durablement 0,75** : à chaque fois qu'il l'atteint, le rehash le redescend aussitôt.

Message clé à faire passer : c'est exactement le résultat théorique du CM, *visible en direct*. Le facteur de charge se comporte comme une dent de scie oscillant entre $\approx 0,4$ (juste après rehash) et 0,875 (juste avant rehash). En moyenne, il reste largement sous 0,75.

Vérification 4 (distribution des seaux, étape 7) – réponses attendues (valeurs approximatives, dépend de `String.hashCode`) :

1. Avec 12 noms sur $m = 16$ ou $m = 32$ cases selon où s'est produit le rehash, **typiquement 1 à 3 seaux avec 2 paires**, les autres avec 0 ou 1.
2. **Plusieurs seaux vides** (attendu : $\alpha < 1$ signifie qu'il y a plus de cases que de paires).
3. Le seau le plus rempli **dépasse rarement 3** avec `String.hashCode` bien distribué sur des noms propres.

Note : si un étudiant obtient une distribution très déséquilibrée (seau à 5, autres vides), c'est instructif : ça montre concrètement pourquoi on doit faire confiance à la fonction de hachage mais se préparer aux collisions. Un bon sujet de discussion.

Partie II – Exercices théoriques : solutions

Exercice 1 – Calcul de positions

Clé	<code>hashCode()</code>	$h(\text{clé}) = \text{hashCode}() \bmod 7$
"Awa Diop"	-2045 123 461	$2\,045\,123\,461 \bmod 7 = 3$
"Moussa Ndiaye"	879 654 325	$879\,654\,325 \bmod 7 = 4$
"Fatou Sall"	1 234 567 892	$1\,234\,567\,892 \bmod 7 = 5$
"Cheikh Ba"	-555 888 777	$555\,888\,777 \bmod 7 = 3$
"Aminata Fall"	42	$42 \bmod 7 = 0$

Collisions : « **Awa Diop** » et « **Cheikh Ba** » partagent $h = 3$.

Piège : ne pas oublier la valeur absolue pour les `hashCode` négatifs. *Piège bis* : $42 \bmod 7 = 0$, pas 5 — souvent mal calculé.

Exercice 2 – Trace de chaînage

Case	Liste chaînée
[0]	Aminata Fall → null
[1]	(vide)
[2]	(vide)
[3]	Awa Diop → Cheikh Ba → null
[4]	Moussa Ndiaye → null
[5]	Fatou Sall → null
[6]	(vide)

Comparaisons pour `get("Aminata Fall")` : **1** (la case [0] ne contient qu'une seule paire).

Piège : certains étudiants mettent Cheikh *avant* Awa en case [3]. Rappeler que `LinkedList.add()` insère en fin de liste, donc l'ordre d'insertion = ordre d'apparition.

Exercice 3 – Trace de sondage linéaire

(a) **Tableau final** :

Case	[0]	[1]	[2]	[3]	[4]	[5]	[6]
Clé	Aminata	–	–	Awa	Moussa	Fatou	Cheikh
<i>h</i> origine	0	–	–	3	4	5	3
Étapes probing	0	–	–	0	0	0	3

Déroulement pas à pas :

- Awa ($h = 3$) : [3] libre → [3].
- Moussa ($h = 4$) : [4] libre → [4].
- Fatou ($h = 5$) : [5] libre → [5].
- Cheikh ($h = 3$) : [3] pris → [4] pris → [5] pris → [6] libre → [6]. **3 étapes de probing.**
- Aminata ($h = 0$) : [0] libre → [0].

(b) **Un seul cluster** de 4 cases contiguës : positions [3], [4], [5], [6].

(Aminata en [0] est isolée, ne forme pas un cluster multi-cases.)

(c) `get("Ibrahima Kane")` avec $h = 3$:

Indices visités : **[3] → [4] → [5] → [6] → [0] → [1]** (arrêt sur case vide).

Ibrahima $\neq$ Awa (en [3]), $\neq$ Moussa ([4]), $\neq$ Fatou ([5]), $\neq$ Cheikh ([6]), $\neq$ Aminata ([0]), puis case [1] *vide* : on sait que la clé n'existe pas.

Résultat : **Absent**, après avoir parcouru **5 cases occupées + 1 case vide**.

Observation importante : la recherche d'absence coûte cher à cause du cluster. C'est exactement la raison pour laquelle on limite $\alpha < 0,75$ avec rehash.

Exercice 4 – Facteur de charge et rehash

(a) Calcul du seuil : $n > 0,75 \times 8 = 6$.

Le rehash se déclenche dès que n dépasse strictement 6, donc à la $k = 7^{\text{ème}}$ insertion.

Précision importante : le test $n/m > 0,75$ est évalué *après* l'insertion. Après la 6^e insertion, $n/m = 6/8 = 0,75$, test strict $>$ faux, pas de rehash. Après la 7^e, $7/8 = 0,875 > 0,75$, rehash.

(b) $m' = 16$ (on double). $\alpha = 7/16 = 0,4375$.

(c) Nouveau seuil : $n > 0,75 \times 16 = 12$. 2^e rehash à $n = 13$, soit **6 insertions supplémentaires** depuis le 1^{er}.

(d) **Taille m après 100 insertions** :

Suite des rehashs :

- $m = 8$, rehash à $n = 7 \rightarrow m = 16$.
- $m = 16$, rehash à $n = 13 \rightarrow m = 32$.
- $m = 32$, rehash à $n = 25 \rightarrow m = 64$ (seuil $0,75 \times 32 = 24$).
- $m = 64$, rehash à $n = 49 \rightarrow m = 128$ (seuil = 48).
- $m = 128$, rehash à $n = 97 \rightarrow m = 256$ (seuil = 96).
- $m = 256$, seuil = $192 > 100$: pas encore de rehash.

Après 100 insertions, $m = \mathbf{256}$, avec $\alpha = 100/256 \approx 0,39$.

Réponse : $m = 256$.

Commentaire : environ $\log_2(100/8) \approx 3,6$ rehashs (arrondi supérieur à 5), chacun coûtant $\Theta(n)$ — mais amortis sur les 100 insertions, le coût par insertion reste $\mathcal{O}(1)$.

Exercice 5 – Contrat equals / hashCode

(a) Contrat Java : Si `a.equals(b)` retourne `true`, alors `a.hashCode() = b.hashCode()` obligatoirement.

La réciproque n'est pas requise : deux objets distincts peuvent avoir le même hashCode (collision autorisée). Mais deux objets considérés égaux *doivent* avoir le même hashCode.

(b) `c1.equals(c2)` vaut **true** (l'égalité porte sur le nom seul, identique). Les hashCode sont **différents** (car `hashCode()` hérité d'`Object` dépend de l'adresse mémoire de chaque objet).

Pourquoi `map.get(c2)` retourne null : `put(c1, "A")` calcule $h_1 = |c_1.hashCode()| \bmod m$ et range la paire dans la case $[h_1]$. Quand on fait `get(c2)`, on calcule $h_2 = |c_2.hashCode()| \bmod m$. Comme `c1.hashCode()  $\neq$  c2.hashCode()`, on a (très probablement) $h_1 \neq h_2$. On cherche dans la case $[h_2]$, qui est vide. Retour `null`, **bien que la clé équivalente existe dans une autre case.**

(c) hashCode() corrigé :

```
@Override
public int hashCode() {
    return nom.hashCode();
}
```

Justification : puisque `equals` compare *uniquement* le nom, le hashCode doit dépendre uniquement du nom. Deux `Contact` avec le même nom ont alors nécessairement le même hashCode, contrat respecté.

Variante acceptable : `return Objects.hash(nom);` (équivalent, style Java moderne).

Variante incorrecte : `return nom.hashCode() + numero.hashCode();` — le numéro influencerait le hashCode alors qu'il n'influence pas `equals`. Deux clés « égales » pourraient avoir des hashCode différents : contrat violé, même bug qu'avant.

Notes pédagogiques pour la plénière

Messages principaux à faire ressortir :

- La partie pratique (étape 6 en particulier) matérialise le comportement *en dent de scie* du facteur de charge. C'est ce qui relie la théorie (l'analyse amortie) à l'observation.
- Le test `testForcerUneCollision` montre que `equals` et `hashCode` jouent des rôles complémentaires : `hashCode` place dans le seau, `equals` identifie la bonne paire *dans* le seau.
- L'exercice 3 (sondage linéaire) matérialise pourquoi le clustering est problématique : la recherche d'absence coûte proportionnel à la taille du cluster, pas à α .

Erreurs fréquentes à anticiper :

- **Pratique :** oubli du cast (`double`) dans le test du facteur de charge $\rightarrow$ jamais de rehash.
- **Pratique :** tests qui plantent avec `ConcurrentModificationException` dans `remove` si on utilise `for-each`.
- **Ex. 1 :** oubli de la valeur absolue sur hashCode négatif ; calcul erroné de $42 \bmod 7$.
- **Ex. 2 :** inversion de l'ordre des clés dans la liste [3].
- **Ex. 3 :** confusion entre h d'origine et position finale après probing. Oubli de compter les étapes du probing de Cheikh.
- **Ex. 4 :** confusion « seuil atteint » ($\alpha = 0,75$) vs « seuil dépassé » ($\alpha > 0,75$ strict).
- **Ex. 5 :** donner un hashCode qui mélange nom + numéro (viole le contrat).